

UdaciHeadline Final Report

Inference optimization study for headline generation

This report summarizes the optimization work completed in the UdaciHeadline notebook. The task was to generate concise news headlines from article summaries, then measure how different inference techniques changed speed, memory use, and headline quality.

The results are based on the saved notebook run using one hundred evaluation examples from the News Category Dataset v3. The run used CUDA on a SageMaker style workspace with four Tesla T4 GPUs, float16 arithmetic, greedy decoding, and a fixed limit of twenty four new tokens per headline.

Methodology

The dataset was filtered to keep rows with both a headline and a short description. Descriptions shorter than thirty characters were removed, then a deterministic shuffle selected the evaluation set. This keeps the comparison fair because every technique uses the same examples and the same generation budget.

The baseline model was Meta Llama 3.2 1B Instruct. For speculative decoding, the larger Meta Llama 3.2 3B Instruct model was measured alone, then measured again with the 1B model as the draft assistant. The notebook used transformers, PyTorch, Accelerate, bitsandbytes, pandas, NumPy, evaluate, rouge score, and the Hugging Face Hub.

Latency was measured per generated headline after warmup. Throughput was calculated from generated tokens per second. Quality was measured with ROUGE 1, ROUGE 2, and ROUGE L against the reference headlines.

Benchmark Results

Table 1 reports the speed and memory measurements. Lower latency is better, while higher throughput is better. Memory is the peak GPU memory reported by the notebook.

Technique	Mean latency s	P99 latency s	Tokens per s	Samples per s	Peak memory MB
Baseline No Cache	0.208	0.409	49.7	4.81	2492
KV Caching	0.176	0.352	59.1	5.67	2495
Pruning 30 percent	0.167	0.388	58.6	6.00	2495
Quantization 4 bit	0.323	0.572	37.9	3.09	1091
Tensor Parallelism	0.251	0.504	41.5	3.98	788
Pipeline Parallelism	0.252	0.504	41.5	3.97	788
Speculative Target Alone	0.415	0.619	30.2	2.41	8934
Speculative Decoding	0.541	0.894	23.3	1.85	8963

Table 2 reports headline quality. The ROUGE values are low in absolute terms because headline writing has many valid phrasings, but the relative changes are still useful.

Technique	ROUGE 1	ROUGE 2	ROUGE L	Examples
Baseline No Cache	0.120	0.020	0.111	100
KV Caching	0.120	0.020	0.110	100
Pruning 30 percent	0.097	0.017	0.088	100
Quantization 4 bit	0.131	0.022	0.115	100
Tensor Parallelism	0.120	0.020	0.110	100
Pipeline Parallelism	0.120	0.020	0.110	100
Speculative Target Alone	0.134	0.021	0.117	100
Speculative Decoding	0.135	0.021	0.118	100

Tradeoff Analysis

Baseline

The baseline without cache averaged 0.208 seconds per headline and produced 49.7 tokens per second. It is useful as a reference, but it repeats attention work during generation and leaves an easy performance gain unused.

KV Caching

KV caching was the strongest practical optimization. Mean latency fell to 0.176 seconds and throughput rose to 59.1 tokens per second. ROUGE L moved from 0.111 to 0.110, which is effectively unchanged. The cost is a small amount of extra cache memory during decoding, but the implementation is simple and already supported by the generation API.

Pruning 30 percent

Pruning showed a low mean latency at 0.167 seconds, but the quality drop is important. ROUGE 1 fell from 0.120 to 0.097 and ROUGE L fell from 0.111 to 0.088. The notebook also notes that dense GPU kernels usually do not exploit the zero weights, so memory stayed near the baseline. This makes pruning a poor choice for this deployment unless it is paired with sparse kernels and more careful recovery training.

Quantization 4 bit

Quantization gave the clearest memory reduction. Peak GPU memory fell to about 1091 MB, less than half the baseline. The tradeoff is speed: mean latency increased to 0.323 seconds and throughput fell to 37.9 tokens per second. Quality was slightly higher in this run, but that should be read as normal evaluation variation rather than proof that 4 bit weights improve headline quality.

Multi GPU Sharding

The tensor and pipeline comparisons used Accelerate device maps for layer sharding. They reduced per GPU memory to about 788 MB, but both were slower than the cached single model path, with mean latency around 0.251 seconds. For a 1B model that already fits on one T4, communication overhead outweighs the memory benefit.

Speculative Decoding

Speculative decoding did not help this workload. The 3B target alone averaged 0.415 seconds, while the assisted run averaged 0.541 seconds. Quality was almost the same, but the short headline length gives the draft and verification process too little room to pay back its overhead. This technique may be more useful for longer generations or with a better matched draft model.

Recommendation

For this headline generation task, KV caching is the recommended production default. It is fast, simple, low risk, and keeps headline quality essentially unchanged. It also requires no model retraining and no special serving hardware beyond the normal generation stack.

Deployment Note

Quantization is the best secondary option when memory or hosting cost is the primary limit. It should be considered for a memory constrained deployment, but not as the first speed optimization for this small model. Pruning should not be deployed from these results because it damaged ROUGE quality without reducing memory. Multi GPU sharding and speculative decoding are also not recommended for the current setup because they add complexity and are slower on the measured workload.